

Commodore 128 Technical Report

Performance

CPU Clock Speeds and Throughput

The Commodore 128 employs two microprocessors: a primary MOS Technology 8502 (an enhanced 6510) and a secondary Zilog Z80A. The 8502 runs at approximately **1 MHz** in standard C128 40-column mode (matching the C64's clock rate) and can double to **2 MHz** in 80-column mode ¹. In practice this means the 8502 is clocked at ~1.02 MHz (NTSC) or ~0.985 MHz (PAL), and up to ~2.04 MHz (NTSC) or ~1.97 MHz (PAL) when the system is in "fast" mode with the 80-column VDC display active ¹. The Z80A co-processor is clocked at **4 MHz**, typical for CP/M machines, but due to the C128's bus design its effective speed is roughly **2 MHz** in operation ² ³ (the shared memory bus and synchronization with video hardware force extra wait states). In other words, the C128's Z80 runs CP/M noticeably slower than a dedicated 4 MHz Z80 system ³.

Despite the modest clock frequencies by modern standards, the C128's CPUs were competitive for mid-1980s home computers. The 2 MHz mode on the 8502 yields nearly twice the throughput of the Commodore 64's 6510 CPU (approx. 2 million operations per second vs. 1 million) in compute-heavy tasks. However, this boost only applies when the 80-column display (VDC) is used or the 40-column screen is disabled – the VIC-IIe video chip otherwise steals cycles, constraining the 8502 to ~1 MHz ¹. The Z80's performance under CP/M was underwhelming; beyond the halved effective clock rate, any I/O or interrupt in CP/M had to hand control to the 8502, creating additional overhead ³. This architectural quirk made the C128's CP/M mode slower in practice than many dedicated CP/M machines, and few users heavily relied on CP/M on this system ³.

Benchmarks and Real-World Application Performance

In general computing tasks (e.g. BASIC execution, productivity software), the C128's 8502 at 2 MHz provided a significant speed-up over the C64. For instance, Commodore BASIC 7.0 on the C128, which is more feature-rich but larger than the C64's BASIC 2.0, ran *slower* than C64 BASIC at 1 MHz, but could exceed it in performance when the 2 MHz fast mode was engaged ⁴. This fast mode benefited number-crunching and logic in applications, provided they could run with the screen blank or on the 80-column display. 128K of RAM also allowed BASIC programs to use separate memory banks for code and variables, reducing garbage collection pauses ⁵. Graphics and gaming performance in native C128 mode saw less improvement, because most C128-specific software (with a few exceptions) did not heavily exploit 2 MHz mode or the 80-column VDC for gameplay. The majority of games ran in C64-compatible mode (1 MHz, 40-column VIC-II) for broader compatibility. Thus, in terms of games, a stock C128 performs similarly to a C64 for most titles. A handful of programs (e.g. some text adventures and productivity tools) offered enhanced C128 versions, using 80-column text or faster disk access, improving the user experience with crisper text and shorter load times.

One area of real improvement was disk I/O and GEOS. The C128's VDC 80-column mode and increased RAM made it a more capable platform for the GEOS graphical OS ⁶. Additionally, when paired with the Commodore 1571 disk drive, the C128 could use a "burst mode" serial transfer that dramatically increased disk loading speed compared to the C64's slow IEC serial rates. This gave the C128 an edge in productivity workflows (faster disk operations, larger documents in memory, etc.). Still, from a modern perspective, all of these 8-bit operations are extremely slow: even at 2 MHz, the 8502 executed on the order of 0.5 MIPS, and disk transfers were in the kilobytes per second. Contemporary reviews noted that the C128 felt faster and more capable than a C64 in programming and office tasks, but it was not in the same performance class as the emerging 16-bit computers.

Comparison with Contemporaries and Modern Insights

To put the C128's performance in context, Table 1 summarizes key specifications of the C128 and two contemporaries (the Apple IIe and IBM PCjr), along with the older C64 for reference:

Table 1 – Commodore 128 vs. 1983–85 Contemporaries

System	CPU (Bits)	Clock Speed	RAM (base)	Notable Video Modes & Graphics
Commodore 128	MOS 8502 (8-bit 6502-family) and Z80A (8-bit)	1 MHz (40-col) / 2 MHz (80-col) ¹ ; Z80 @ 4 MHz (effective ~2 MHz) ²	128 KB	40×25 text/graphics: 320×200 up to 16 colors (VIC-IIe); 80×25 text: 640×200, 16-color text or mono bitmap (VDC) ⁷ ⁸ ; hardware sprites, scrolling, SID audio chip
Commodore 64	MOS 6510 (8-bit 6502-family)	~1 MHz (0.985 PAL / 1.023 NTSC) ⁹	64 KB	40×25 text; 320×200 graphics (16 colors) with sprites ¹⁰ ; sound via SID chip (3 voices)
Apple IIe	MOS 6502 / 65C02 (8-bit)	1.023 MHz ¹¹	64 KB (base) ¹²	40×24 text (black & white); optional 80-col text; 280×192 graphics (6 colors) or 560×192 (with extension, 15 colors composite) ¹³ ; no sprite hardware, 1-bit beeper sound
IBM PCjr	Intel 8088 (16-bit ext. bus 8-bit)	4.77 MHz ¹⁴	128 KB (max onboard) ¹⁵ ¹⁶	40×25 text; CGA-compatible graphics: e.g. 320×200 in 16 colors, 640×200 in 4 colors ¹⁷ ; no sprites (soft graphics), TI SN76489 PSG sound (3-voice)

Performance Notes: In raw CPU power, the C128's 2 MHz 8502 could outperform the Apple IIe's 1 MHz 6502 in general computation by roughly 2×, and approach the IBM PCjr's 4.77 MHz 8088 in some tasks. The PCjr's 8088, although higher clocked and a 16-bit processor, was hobbled by wait states from its shared video memory; in fact, due to memory contention, the PCjr often ran *slower* than an IBM PC despite the same clock – designers noted it might incur up to six extra cycles for memory accesses ¹⁶. The Apple IIe's

lack of graphics acceleration meant its 1 MHz CPU had to handle all drawing, whereas the C128/C64's VIC-II chip offloaded graphics like sprites and scrolling to hardware. Thus, in games and graphics demos, the C128 (even in 1 MHz mode) could produce smoother motion and richer visuals than an Apple IIe at 1 MHz or a PCjr using CPU-driven graphics.

From a modern computing paradigm standpoint, the C128 exemplified an early form of **heterogeneous computing** and **backward compatibility** for performance: it had dual CPUs with different instruction sets for different tasks (somewhat analogous to modern systems with specialized co-processors or multi-core designs, though the 8502 and Z80 ran in *alternate* modes rather than concurrently). Its use of specialized chips for video (VIC-IIe and VDC) and sound (SID) is a precursor to today's GPUs and audio DSPs that offload work from the CPU. The C128 taught that raw clock speed isn't the only determinant of performance – memory architecture and specialized hardware matter. Indeed, the 4 MHz Z80 did not make the C128 the fastest CP/M machine because the **system architecture** (bus sharing, I/O delegation to the 8502, etc.) bottlenecked it ³. This echoes modern insights: effective performance depends on memory bandwidth, parallelism, and system balance, not just CPU GHz. In summary, compared to its peers, the C128 provided strong performance in 8-bit computing tasks, leveraging its dual speeds and chips – but like all 1980s micros, it is minuscule against today's gigahertz multi-core processors.

Architecture

Dual-CPU Design and Operating Modes

At the heart of the C128 is a two-CPU configuration: the MOS 8502 is the **primary processor** for Commodore native modes, while the Zilog Z80A is included mainly to run CP/M software ¹⁸. Only one CPU is active at a time, and a custom Memory Management Unit (MMU, MOS 8722) coordinates which CPU controls the bus ¹⁹. On power-up, the Z80 takes brief control first – it checks for a CP/M boot disk, and if found, will continue running CP/M; otherwise it hands off the system to the 8502 which boots into Commodore BASIC 7.0 (C128 mode) ²⁰ ²¹. The user can manually force C64-mode (for 99% compatibility with C64 software) by holding the C64 “Commodore” key at boot or by issuing the `GO 64` command in BASIC, which triggers a warm reset into the C64 environment. In all modes, the inactive CPU is effectively tri-stated or held in reset so it does not interfere with the bus.

Boot/Mode Selection: The mode selection via the Z80 at startup is an elegant solution to multi-environment support. The Z80 checks for a CP/M disk and also whether any C64/C128 cartridge is present or the C64-mode key held, then vectors to the appropriate startup routine ²⁰. CP/M mode actually utilizes *both* CPUs in a coordinated way – the Z80 executes CP/M BIOS and programs, but the 8502 is used as an intelligent I/O controller for Commodore-specific hardware access and interrupts ³. This design was necessary because the C128's I/O chips and video were designed for 65xx CPUs; the Z80 alone could not easily drive all peripherals. While unusual, this dual-CPU setup (with different ISAs) provided tremendous software library breadth: the C128 could natively run C128 software, virtually all C64 software, and a large portion of CP/M 3.0 software ¹⁸ ²².

Memory Architecture and Banking System

The Commodore 128 is equipped with **128 KB of RAM** – double the 64 KB of the C64. However, the 8502 is an 8-bit processor with a 16-bit address bus, inherently only able to address 64 KB at a time. Commodore solved this by implementing a bank-switched memory architecture using the MMU (8722 chip) ²³ ¹⁹. The

RAM is divided into two 64 KB **banks** (Bank 0 and Bank 1). The MMU can switch which bank the CPU sees in its 0x0000–0xFFFF address space at any given time, and also map ROM in and out as needed. In C128 native mode, BASIC programs and data can be split between banks – for example, program code in one bank and variables in the other – effectively allowing much larger BASIC programs than the 64K limit ⁵. The BANK command in BASIC 7.0 lets programs select memory banks for different purposes.

To facilitate communication between banks and to handle special memory areas, the C128 designates certain regions as **“common” or “shared” memory**. Specifically, the lowest 1 KB of memory (\$0000–\$03FF, which includes the zero-page and hardware stack) and the highest 256 bytes (\$FF00–\$FFFF, which holds system vectors and MMU registers) are always accessed from the *opposite* bank than the rest of memory ²⁴. In other words, these regions are shared between Bank 0 and Bank 1 – if the 8502 is currently using Bank 0 for the main 64K, any reads/writes to the shared areas actually go to Bank 1, and vice-versa ²⁵. This clever scheme means critical data (like zero-page variables, stack, and interrupt vectors) can remain common and easily accessible regardless of the active bank, simplifying cross-bank operation. The VIC-IIe video chip can also be pointed to either bank for its video memory, and there are two sets of color RAM (one per bank) to allow the VIC-IIe to fetch character colors properly when the active bank is switched ²⁶.

ROM and Address Map: The C128 contains a total of 72 KB of ROM ²⁷, including BASIC 7.0 (around 32 KB), the C128 KERNAL OS, a machine language monitor, plus the C64’s BASIC 2.0 and KERNAL for C64 mode. These ROMs are banked in and out of the address space via the MMU and a processor port. In C64 mode, the memory map is set to mimic a C64: only 64K is accessible (the extra RAM is hidden), and the C64’s 8 KB BASIC and 8 KB KERNAL ROMs are mapped in at their typical addresses (\$A000–\$BFFF and \$E000–\$FFFF, respectively), along with the 4 KB character ROM at \$D000. In C128 mode, the larger BASIC 7.0 ROM (32 KB) and C128 KERNAL (16 KB) are mapped in, and memory banking is enabled for RAM above 64K. The MMU can also relocate the 8502’s zero-page (\$00–\$FF) and stack (\$100–\$1FF) to any 256-byte page in memory ²⁸, a powerful feature that allows optimizing assembly language routines by giving them a custom zero-page (since 6502-family CPUs have special fast addressing for page 0). Overall, the memory architecture is quite complex for an 8-bit machine – it was designed to maximize flexibility and make the best use of the full 128 KB, at the cost of added complexity in low-level programming.

Video Hardware: VIC-IIe and VDC

One of the C128’s unique features is its **dual video chips**, which provide two separate display outputs with different capabilities. The first is the **MOS 8564/8566 VIC-IIe**, essentially an improved version of the C64’s VIC-II graphics chip. The second is the **MOS 8563 VDC** (or 8568 in later C128DCR models), an 80-column color video controller.

- **VIC-IIe (40-column)** – This chip is backward-compatible with the Commodore 64’s graphics. It generates the **40×25 character text mode** and graphics modes on the C128’s composite or RF video output. The VIC-IIe supports a bitmap resolution of **320×200 pixels** with 16 colors (15 colors + background) in standard high-res mode, or **160×200** with 16 colors in multicolor mode (where each 2×1 block of pixels shares a color) ²⁹. It also provides the famous hardware **sprites** (8 sprites, 24×21 pixels, with collision detection) and smooth scrolling features that made the C64 a capable games machine ³⁰. The “e” variant of VIC-II in the C128 has a few subtle differences from the C64’s VIC-II: it can access two 16KB video RAM regions (since the C128 has two banks) and supports an extra text mode feature where an additional 1K of color RAM is available to the programmer (via multiplexing through memory address 1) ³¹. But fundamentally, VIC-IIe produces the same resolutions and color

limitations as in the C64. Importantly, the VIC-IIe is tied to the system bus – it **steals CPU cycles** while drawing the 40-column screen. This is why the 8502 cannot run at 2 MHz in 40-column mode; the VIC chip's timing expects a 1 MHz bus and takes half the cycles for video DMA. If a program does attempt 2 MHz with VIC-II active, the display will corrupt or blank ³². Some software creatively toggled the 2 MHz mode during the vertical blank or border areas to get a slight speed boost without disrupting the visible display ³².

- **VDC (80-column)** – The 8563 VDC is an entirely separate video controller that drives the C128's 80-column RGBI output (digital RGB with intensity, compatible with CGA monitors). It has its **own dedicated video RAM** (16 KB by default, expandable to 64 KB) and does not contend with the 8502 for main memory. This independence is what allows the 8502 to run at 2 MHz when using the 80-column screen – the VDC reads from its private VRAM instead of stealing system bus cycles. The VDC's standard text mode is **80×25 characters** (each char 8×8 pixel matrix, for an effective 640×200 resolution text screen) with support for attributes like foreground/background color (16-color palette), underline and blink (it emulates an ADM-3A terminal with some enhancements) ⁸ ³³. Graphically, the VDC was not as straightforward to use as the VIC: it can generate high-resolution **bitmapped graphics** (up to 640×200, and up to 16 colors if VRAM is expanded to 64 KB) ³⁴, but there is no direct memory-mapped framebuffer. Instead, the CPU must program the VDC via I/O registers – a port at \$D600/\$D601 is used to index and access the VDC's memory and registers ³⁵. This means drawing to the 80-column screen in software is relatively slow, as each pixel or character operation goes through a register read/write interface (much like early PC graphics cards). The VDC does include a limited **blitter**/block copy capability ³⁶ which can accelerate certain operations (e.g., fast text scrolling by copying memory). In practice, few games utilized the 80-column VDC for graphics, but it excelled at displaying sharp text for programming, word processing, or CP/M use. Many CP/M programs and business software expected an 80-column display, and the C128 could provide that (a major advantage over the C64's 40-column limit).

The presence of two different video outputs meant a C128 could even drive *two monitors* simultaneously (one showing 40-col VIC output, another the 80-col VDC output). There are demos and Easter eggs that put different content on each screen. However, the system has only one SID sound chip and one set of I/O, so typically software uses one display at a time. From an architectural perspective, the dual-video system is notable: the VIC-IIe is on the main system bus (mapped into memory at \$D000–DFFF like on the C64) and shares RAM with the CPU, whereas the VDC is more like an external device on its own bus, accessed via registers. Modern computers similarly have both integrated graphics (sharing main RAM) and discrete video cards with dedicated VRAM – the C128 had both in one machine, in 1985.

Bus Structure, DMA, and Peripherals

The C128's system bus is an evolution of the C64's architecture. The 8502 CPU has a built-in I/O port (at memory address \$00/\$01) that, as in the C64, is used to control memory bank configuration (enabling or disabling ROM, I/O, etc., in the address map). The dual CPUs required bus arbitration logic: a hardware PLA and the MMU coordinate which CPU is driving the address/data lines. When the Z80 is active (CP/M mode), the 8502 is put into a wait state (or vice versa). A latch (U2) and buffer (U13) in the data lines help interface the Z80 to the 8502's bus, as the Z80's control signals differ (it has separate read/write and requires an address/data multiplexing for its DRAM access) ³⁷ ³⁸. In essence, the C128's bus is designed around the 6502-style timing, so the Z80 must adapt to that, contributing to wait states for synchronization.

DMA and Memory Access: There is no dedicated bus-master DMA controller in the C128 aside from the video chips performing their memory access. The VIC-IIe performs **DMA steals** on the bus for each video raster line to fetch character or bitmap data from main RAM (this is what enforces the 1 MHz CPU limit when VIC is on). The VDC, having its own RAM, does not steal the main bus; instead, if the CPU needs to manipulate VDC memory, it does so via the I/O port (which is effectively programmed I/O, not DMA). There was an optional external RAM Expansion Unit (1700/1750 REU, up to 512 KB) that connected via the cartridge port – the REU contained its own DMA logic that could rapidly swap blocks of data with the C128's RAM when triggered, all under software control. This is somewhat analogous to an external co-processor doing DMA transfers. While not part of the base C128, it's worth noting since it shows the system's extensibility.

Peripheral Interface: The C128 retains Commodore's standard peripheral ports, most of which are carried over from the C64: an **IEC serial bus** port for disk drives and printers, the datasette **cassette tape port**, two Atari-compatible **joystick ports**, an **audio/video output**, an **RF TV modulator**, and the **cartridge/expansion port**. The **IEC serial bus** was enhanced in the C128/1571 combination by allowing a **burst mode**: when using a 1571 disk drive, the C128 can transfer data roughly an order of magnitude faster than the C64 could with a 1541. This is achieved by using the built-in serial shift registers of the CIA chip to clock data at 2 MHz between computer and drive (the 1571's on-board 6502 and VIA chip handle the other end). In C64 mode, or with older drives, the bus reverts to the slow bit-banged serial (about 300 bytes/second). From a modern standpoint, this is still painfully slow, but at the time, C128 owners appreciated the much quicker disk loads with a 1571 or 1581 drive.

Another addition on the C128 is the **RGBI video port** for the 80-column display, which outputs TTL-level RGB signals plus intensity – the same format used by IBM CGA monitors. This was a departure from the C64 which had only composite and S-Video output. It indicated Commodore's intent for the C128 to be useful as a small business machine (many came with a "green screen" or color RGB monitor for crisp text).

Despite the new hardware, the C128 maintained impressive backward compatibility. In **C64 mode**, nearly all the C64's quirks are replicated. The same VIC-II and SID chips are present, the memory map mirrors a C64, and even odd behaviors (like reading unconnected I/O lines) act similarly. A few minor differences existed – for example, the 8502's extra I/O port bit was used to sense the Caps Lock key on the C128 (since the C128 had a software-readable Caps Lock instead of the C64's simple Shift Lock) ³⁹. A very small number of C64 programs that peered into the 6510 port or relied on exact hardware faults would fail on a C128, but these were rare. Overall, the C128's bus and I/O design achieved the goal of a system with a **dual personality**: a modernized C64 on one hand and a CP/M-capable 128K system on the other, with minimal compromise between the two.

User Port Details

Pinout and Electrical Specifications

The **user port** on the C128 is the same 24-pin edge connector found on the C64 ⁴⁰, providing a general-purpose 8-bit parallel I/O interface plus serial handshake lines and power. It is essentially an exposed I/O port of one of the C128's two CIA (Complex Interface Adapter) chips. Table 2 shows the pin assignments for the user port:

Table 2 – C128/C64 User Port Pinout ⁴¹ ⁴²

Pin (Edge)	Signal	Description
1, A, N	GND	Ground (common ground on multiple pins)
2	+5V	+5 V DC supply (approx 100 mA max) ⁴¹
3	/RESET	Reset line (input to reset C128, and outputs system reset to devices)
4	CNT1	Timer Counter 1 input (from CIA#1)
5	SP1	Serial Port 1 (CIA#1 serial I/O line)
6	CNT2	Timer Counter 2 input (from CIA#2)
7	SP2	Serial Port 2 (CIA#2 serial I/O line)
8	/PC2	Peripheral Control 2 (handshake line from CIA#2)
9	ATN	Attention detect (input from IEC bus, on C64 tied to serial bus ATN)
10, 11	9VAC	9 VAC from power supply (two pins for both phases, ~100 mA max each) ⁴³
12	GND	Ground
B	/FLAG2	Flag 2 (input line to CIA#2, often used for modem carrier detect or general input) ⁴⁴
C, D, E, F, H, J, K, L	PB0–PB7	Parallel data bits 0–7 (bidirectional lines from CIA#2 Port B) ⁴⁵
M	PA2	Peripheral A2 (an additional I/O line from CIA#2 Port A, used as TxD for serial) ⁴⁶

All signals are **TTL-level** (0–5 V logic). Notably, the user port provides the necessary lines to implement an **RS-232 serial interface**, but without proper level shifting. The TxD and RxD lines for RS-232 are mapped onto the CIA lines: the C128's KERNAL uses **PA2 (pin M)** as transmit data and **PB0 (pin C) / FLAG2 (pin B)** as receive data (these two pins must be tied together for RXD) ⁴⁷. However, these signals are *inverted* TTL relative to true RS-232. In other words, a logic HIGH on the user port corresponds to a negative voltage in real RS-232 convention. **The user port does not output ±12 V signals** – it outputs 0 V for logical “1” (mark) and +5 V for logical “0” (space) on TxD, inverted from standard. Thus, an external level shifter or adapter (such as a MAX232 chip) is required to connect the user port to standard RS-232 devices ⁴⁸. Electrical current is limited; the +5 V pin can source about 100 mA, and drawing more (for example, to power interface boards) can overload the C128's internal fuse or regulator. The **9 VAC** pins provide the AC from the transformer (typically used to derive +/-12 V in some expansions, but not commonly used directly by hobby projects due to the need for additional rectification and regulation).

Typical Use Cases (Historical)

In the Commodore 64/128 era, the user port was a versatile connector for adding peripherals. Common uses included:

- **Modems and Serial Communications:** Commodore's own modems (e.g., the 1650, 1660, 1670 models) plugged directly into the user port, leveraging the RS-232 signals (with appropriate level shifting in the modem cartridge). These modems were limited to low baud rates (typically 300–1200 baud by KERNAL routines, though later software hacks achieved 2400+ baud). Third-party RS-232 adapters were also popular, allowing the connection of standard RS-232 devices like serial printers or communications terminals. Users could connect terminals or even other computers via serial cables. The C128's user port RS-232 implementation was "software emulated" via CIA timers/shift registers, so it was not as efficient as a hardware UART, but it sufficed for dial-up networking of the day ⁴⁸.
- **Printers and Parallel I/O:** Some interfaces provided a Centronics parallel printer port through the user port, since Commodore's serial IEC was notoriously slow for printing. These devices would use the 8 data lines (PB0–PB7) as a byte-wide output to send ASCII to printers, with a couple of lines (like /FLAG2 or /PC2) used for handshake (busy/ack). Users also built custom circuits to interface the user port to IEEE-488 (the parallel bus used by older Commodore PET drives) and other parallel devices.
- **Home Automation and Control:** The user port's 8-bit output could directly drive simple electronics (with proper buffering). Hobbyists used it to interface with relays, lights, and sensors for home automation projects in the 1980s. For example, one could control a set of house lights or read switches by connecting to the user port lines. Magazines like *Compute!* and *The Transactor* published projects using the user port to control robots, thermostats, alarms, etc. The port was essentially a built-in simple **GPIO (General Purpose I/O)** interface decades before Arduino/Raspberry Pi made GPIO commonplace.
- **Inter-computer Links:** Even back in the day, people connected C64/128s to other systems via the user port. Commodore published a scheme for "Easy Intercomputer Connection" wherein two Commodores' user ports could be linked for data transfer ⁴⁹. There were also cables to connect the user port to an IBM PC's serial port (with level conversion) to allow data transfer programs to shuttle files between the two (this was useful for uploading C64 disk images, etc., in later years).

Modern Interfacing Potential and Challenges

Today, the C128/C64 user port remains an entry point for hardware hackers to breathe new life into these vintage machines. Modern devices that interface through the user port include **Wi-Fi modems** (microcontroller-based devices that emulate phone modems but connect to the internet) ⁵⁰, **microcontroller kits** (like Arduino-based projects that let the Commodore control external sensors or LEDs), and even **USB bridges**. For instance, one can connect an FT232R or similar USB-serial adapter to the user port to allow a C128 to communicate with a PC via USB serial. This essentially gives the C128 a "virtual COM port" to a modern computer for data transfer or using the C128 as a terminal. Likewise, hobbyists have connected Raspberry Pi or Arduino boards through level shifters to act as disk drive emulators, network interfaces, or co-processors.

When doing this, a few technical challenges must be addressed:

- **Voltage Level & Inversion:** As mentioned, the user port I/O is 5 V TTL. Many modern microcontrollers use 3.3 V logic, so level shifting is needed (simple resistor dividers or buffer ICs can be used to step 5 V down to 3.3 V, and 3.3 V up to 5 V if needed, though often 3.3 V is read as a high by a 5 V TTL input). Additionally, the inversion of TxD/RxD means software on the C128 either must invert the bits in software or an inverting level shifter must be used. Projects like the “UP9600” hack use the CIA’s serial shift register in a mode that can achieve 9600 baud by exploiting timing, but this still requires an inverter for proper RS232 logic levels ⁵¹ ⁵² . Off-the-shelf modules (MAX232-based) are commonly used to get standard RS-232 from the user port ⁵³ .
- **Current Limits:** The 5 V @100 mA limit means one cannot power power-hungry devices from the user port. For example, a Wi-Fi modem for the C64 typically uses a serial Wi-Fi module (like ESP8266) which can draw near 200–300 mA when transmitting – such devices must use an external power source or be very carefully designed to stay within limits. Drawing too much from the user port will blow the fuse in the C128’s power circuit or cause voltage drop resets.
- **Signal Integrity:** The user port was not originally designed for high-speed data transfer – signals are just TTL lines running along a ribbon cable or PCB header. At higher baud rates or when interfacing with fast logic, care must be taken to avoid noise. Also, the C128’s grounding scheme (multiple GND pins) should be used to ensure a good return path, especially if the connected cable is long.

Despite these challenges, the user port provides a **robust expansion interface**. It effectively gives the modern user a way to inject or extract data from a C128 without modifying the system. For example, enthusiasts have connected Arduino-based ADC/DAC boards to sample and generate signals under C128 control, or connected the C128 to the internet by linking the user port to a Wi-Fi microcontroller that interprets AT-style modem commands ⁵⁰ . The C128 issues standard modem “AT” commands to the user port, and the Wi-Fi device responds by connecting to Telnet BBSes or other services – essentially repurposing the user port for modern networking. Another project uses the user port to bit-bang a protocol to read USB flash drives via a microcontroller, illustrating that, in concept, even USB devices could be mediated (though the C128’s CPU is very slow for USB speeds, so this is limited).

In summary, the C128’s user port is a simple yet powerful 8-bit parallel/serial interface that has been used for everything from connecting **homebrew hardware** in the 1980s to bridging to the **IoT world** in the 2020s. The main technical caveats are the **TTL 5V logic** (and inverted RS-232 signaling) ⁴⁸ , which require appropriate interface circuitry to talk to real RS-232 or modern 3.3V devices, and the speed limitations of bit-banged serial. With those addressed, the user port remains a gateway for expanding the C128’s capabilities well beyond what was imagined when it was designed.

Sources: Primary specifications and technical details were obtained from Commodore 128 official documentation and retrocomputing references, including the C128 Programmers Reference Guide, *Mapping the Commodore 128*, and verified summary resources ¹ ³ ⁵⁴ ²⁴ ⁵⁵ ¹⁶ . These provide insight into the C128’s CPU speeds, memory banking, dual-video architecture, and I/O ports as compared to its contemporaries and from a modern perspective. The user port pinout and voltage information are from the C64/C128 User’s Guide and hardware pinout references ⁴¹ ⁴² ⁴⁸ .

1 7 29 30 34 **Commodore 128 System Info**

<https://www.vgmuseum.com/systems/c128/>

2 3 4 5 6 8 18 19 20 21 22 23 24 25 26 27 28 31 32 33 35 36 39 54 **Commodore 128 - Wikipedia**

https://en.wikipedia.org/wiki/Commodore_128

9 10 **Commodore C-64**

http://oldcomputermuseum.com/commodore_c64.html

11 12 13 **Apple IIe - Wikipedia**

https://en.wikipedia.org/wiki/Apple_IIe

14 15 16 17 **IBM PCjr - Wikipedia**

https://en.wikipedia.org/wiki/IBM_PCjr

37 38 **Full text of "Troubleshooting and repairing your Commodore 128"**

https://archive.org/stream/troubleshooting_and_repairing_your_commodore_128/troubleshooting_and_repairing_your_commodore_128_djvu.txt

40 49 55 **User Port - C64-Wiki**

https://www.c64-wiki.com/wiki/User_Port

41 42 43 44 45 46 **Commodore C64 User Port · AllPinouts**

https://allpinouts.org/pinouts/connectors/cartridges_expansions/commodore-c64-user-port/

47 48 **Commodore C64 RS232 User Port · AllPinouts**

<https://allpinouts.org/pinouts/connectors/serial/commodore-c64-rs232-user-port/>

50 **DIY C64 WiFi modem - We are All Geeks! - WordPress.com**

<https://wereallgeeks.wordpress.com/2024/02/13/c64wifimodem/>

51 **UP9600: How to Bit-Bang 9600 Baud RS-232 on the C64**

<https://www.pagetable.com/?p=1656>

52 **User port pinout for rs-232 serial. - Denial - Sleeping Elephant**

<http://sleepingelephant.com/ipw-web/bulletin/bb/viewtopic.php?t=7501>

53 **DIY RS-232 Interface for Commodore C64 for under \$15 - Biosrhythm**

<https://biosrhythm.com/?p=1136>